



The **while** Loop

Keep Going Until It's Done!

Presented by: Amit Roy
MSc student at IIT

What Exactly is **while**?

I was listening to music **while** doing my homework.

- **Condition:** doing my homework.
- **Statement:** I was listening to music

What Exactly is a **while** Loop?

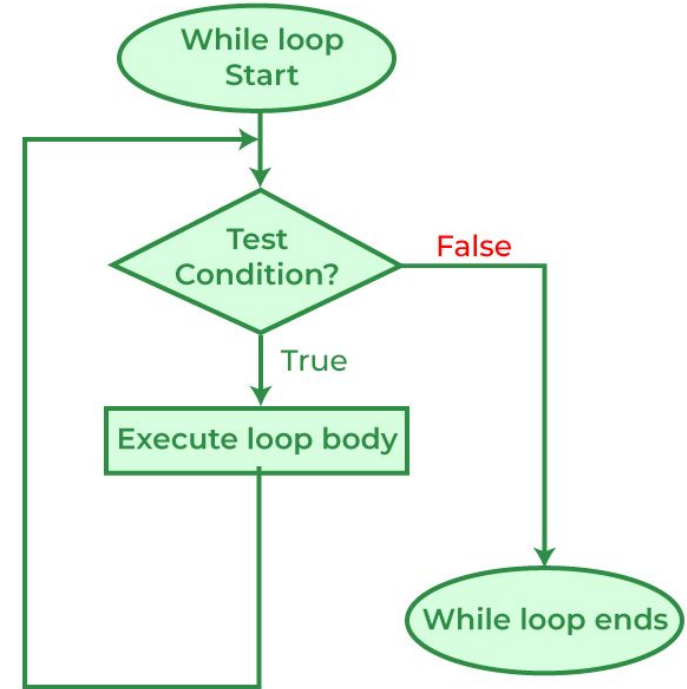
- Think of it like waiting for a bus.
- Condition: 'I will wait as long as the bus hasn't arrived.'
- As long as condition is True (no bus) → you keep waiting (looping).
- When condition becomes False (bus arrives) → you stop waiting!



How the Computer Reads It

1. Check the condition.
2. If it's True, run the code inside.
3. Go back up and check again.
4. If it's False, skip the code and exit!

```
while condition:  
    # Some statement to be executed
```



The 3 Golden Rules!

Every working while loop needs these 3 parts:

1

Initialize

Where do we start?

2

Condition

When do we stop?

3

Update

How do we move forward?

Practice 5.1: Counting From 1 to 5



```
# 1. Initialize
```

```
count = 1
```

```
# 2. Condition
```

```
while count <= 5:
```

```
    print(f"Count: {count}")
```

```
# 3. Update
```

```
    count = count + 1
```

Fun Example: Countdown! 🚀

Problem 5.2: Let's launch a rocket by counting backward from 10. (10 to 1)



```
number = 10
print("Countdown:")

while number > 0:
    print(number)
    # Counting down!
    number = number - 1

print("Blast off! 🚀")
```



10
9
8
7
6
5
4
3
2
1

Class Task 1:

Q . Write for loop statement to print the following series:

10, 20, 30 300

Class Task 1:

Q . Write for loop statement to print the following series:

10, 20, 30 300



```
number = 10
while number <=300:
    print(number)
    # Counting down!
    number = number + 10
```


When to use which loop?

for Loop

When you know exactly how many times to repeat.



"Lift the weight 10 times"

VS

while Loop

When you don't know how many times, just a condition.



"Keep driving until fuel is empty."

For loop vs While loop



```
# Using for loop when you know the exact number of iterations
print("Using FOR loop - counting 1 to 5:")
for i in range(1, 6):          # We know exactly 5 iterations
    print(f"Number: {i}")
```



```
# Using while loop for the same task (less common for counting)
print("Using WHILE loop - counting 1 to 5:")
counter = 1                    # Initialize
while counter <= 5:            # Test condition
    print(f"Number: {counter}") # Loop body
    counter = counter + 1       # Update counter
```

Fun Example: Sum Calculator!

Problem 5.4: Calculate sum of the numbers until user enter 0

Fun Example: Sum Calculator!

Problem 5.4: Calculate sum of the numbers until user enter 0



```
total = 0
number = 1          # Initialized with 1 just to start the loop

print("Enter numbers to add (enter 0 to stop):")

while number != 0:   # Condition: Stop the loop if user enters 0
    number = int(input("Enter a number: "))

    if number != 0:   # Only add if the number is not 0
        total = total + number
        print(f"Running total: {total}")

print(f"Final sum: {total}")
```



DANGER: Infinite Loop

What happens if you forget Rule #3 (The Update)?

- The condition never becomes False.
- The loop runs FOREVER.
- Your program crashes/hangs!

```
count = 1
while count <= 5:
    print(count)
    # MISSING: count = count + 1
    # Prints forever! 💀
```



RUNNING FOREVER...



DANGER: Infinite Loop

Common Mistake

- Forgetting to update the control variable
- Updating in the wrong direction (decreasing when you should increase)
- Using the wrong comparison operator in the condition

Loop Control: Rules Breakers



break

The Emergency Exit.

Stops the loop completely, no matter what the condition is, and jumps out.



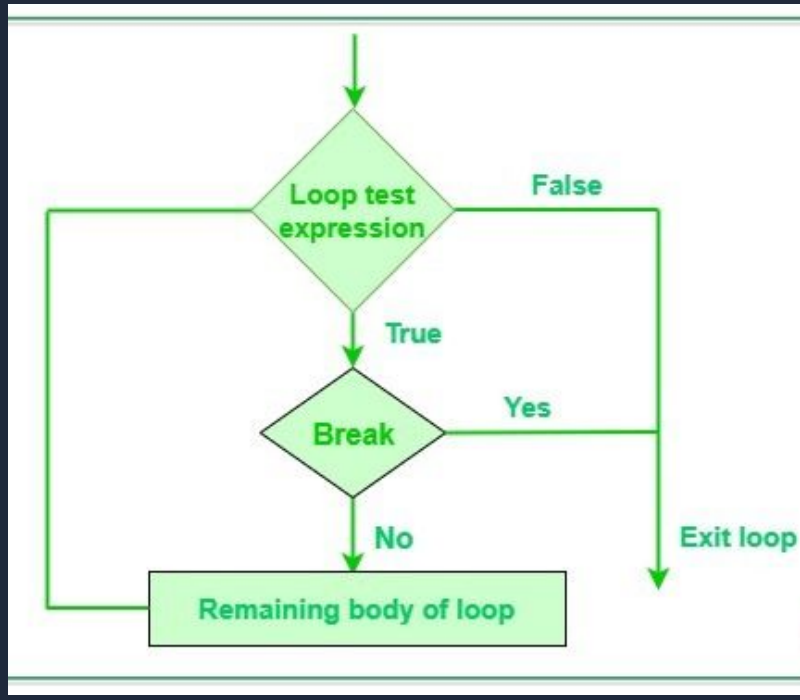
continue

The Skip Button.

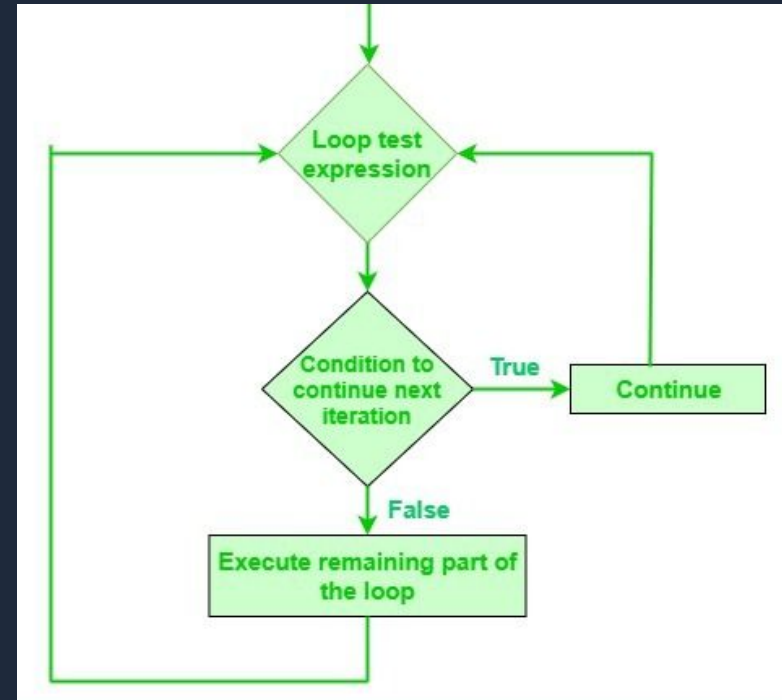
Skips the rest of the current round and jumps right back to the top condition check.

Loop Control: Rules Breakers

break



continue



Loop Control: Rules Breakers

break

```
count = 1

while count <= 5:
    if count == 3:
        break    # loop stop here
    print(count)
    count += 1
```

continue

```
count = 0

while count < 5:
    count += 1
    if count == 3:
        continue # 3 skip
    print(count)
```

Real-Life: Password Checker

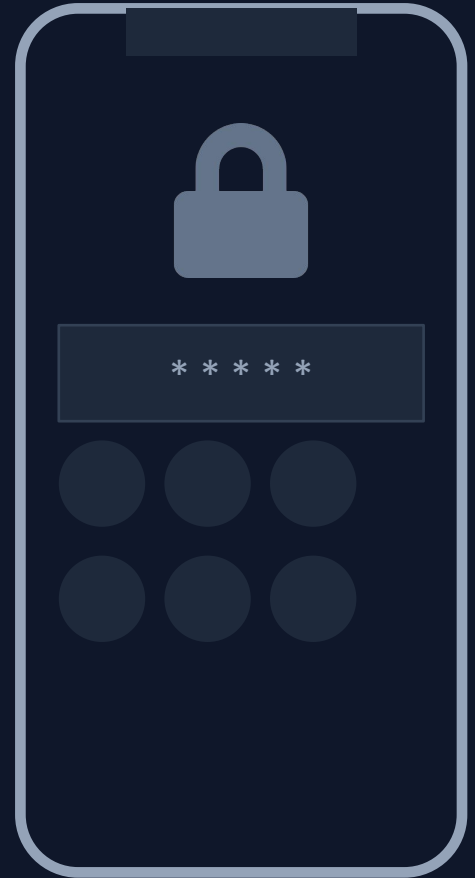
5.8 A common use: force a user to enter valid information. It will keep asking until correct!

```
correct_pwd = "python123"
user_pwd = ""

while user_pwd != correct_pwd:
    user_pwd = input("Enter password: ")

    if user_pwd != correct_pwd:
        print("Wrong! Try again.")

print("Access granted! )
```





Any Questions?

Thank you for listening!

Let's Practice

1. Simple Countdown

- Write a program that asks the user for a starting number and counts down to 1, printing each number.

2. Even Number Counter

- Use a while loop to print all even numbers from 2 to 20.

3. Positive Number Sum

- Keep asking the user for positive numbers and add them to a total. Stop when they enter -1 and display the final sum.

Let's Practice

4. Number Guessing Game

- Pick a secret number (like 7). Keep asking the user to guess until they get it right. Tell them "Too high" or "Too low" for wrong guesses.

5. Multiplication Practice

- Ask the user to solve multiplication problems (like 3×4). Keep giving new problems until they get one wrong. Count how many they got right.

6. Power Calculator

- Ask for a base number. Use a while loop to calculate and print its powers (base, base, base, etc.) until the result exceeds 1000.